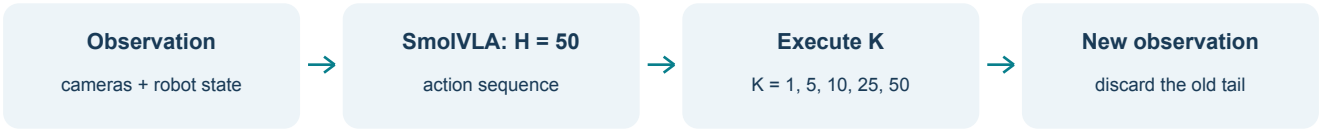
VLA / LIBERO · AN EXPERIMENT IN FEEDBACK CONTROL

How often should a robot replan?

A compact study: 30 episodes across three tasks

Objective: measure how the number of actions K between observation updates affects success, motion and computational cost for one pretrained model. Five preliminary smoke-test episodes are excluded from the results in this report.



One instruction per episode; repeat until success or the step limit.

What the model predicts

SmolVLA receives images from two cameras, the robot state and a language instruction. Its action expert uses flow matching to predict a sequence of 50 actions. Each action specifies relative translation and rotation of the end effector, plus a gripper command. The controller executes the first K actions, obtains a fresh observation and replaces the remaining plan.

Tasks and initial situations

Task	LIBERO suite	Limit
Soup can into basket	libero_object	280 steps
Black bowl onto plate	libero_spatial	280 steps
Open the middle drawer	libero_goal	300 steps

3 tasks × 2 states (0, 1) × 1 seed (11) × 5 values of K = 30 episodes. All K values are evaluated from each initial state. Identical physical states and initial observations are verified using SHA-256 hashes.

Controlled conditions

H = 50; 10 generation iterations; 20 Hz in simulation; batch = 1; precision fp32; 10 settling steps; 2 warmup calls. Camera images are 256 × 256 before the checkpoint's standard preprocessing. The model, normalization, GPU and video recording are identical. Blocks and K values are shuffled using seed 20261017. Model noise is tied to the policy seed and simulation step.

K changes the length of the executed prefix. The prediction horizon H stays fixed; a single action is not repeated K times. The simulator waits for model inference to finish.

Results: task completion

Success means satisfying LIBERO's native criterion before the step limit. The table reports successful episodes divided by all episodes; each K has six episodes.

K	Soup can	Bowl	Drawer	Overall	95% interval
1	0/2	1/2	1/2	2/6	0-67%
5	2/2	1/2	2/2	5/6	67-100%
10	2/2	0/2	2/2	4/6	67-67%
25	1/2	0/2	1/2	2/6	0-67%
50	1/2	1/2	0/2	2/6	0-67%

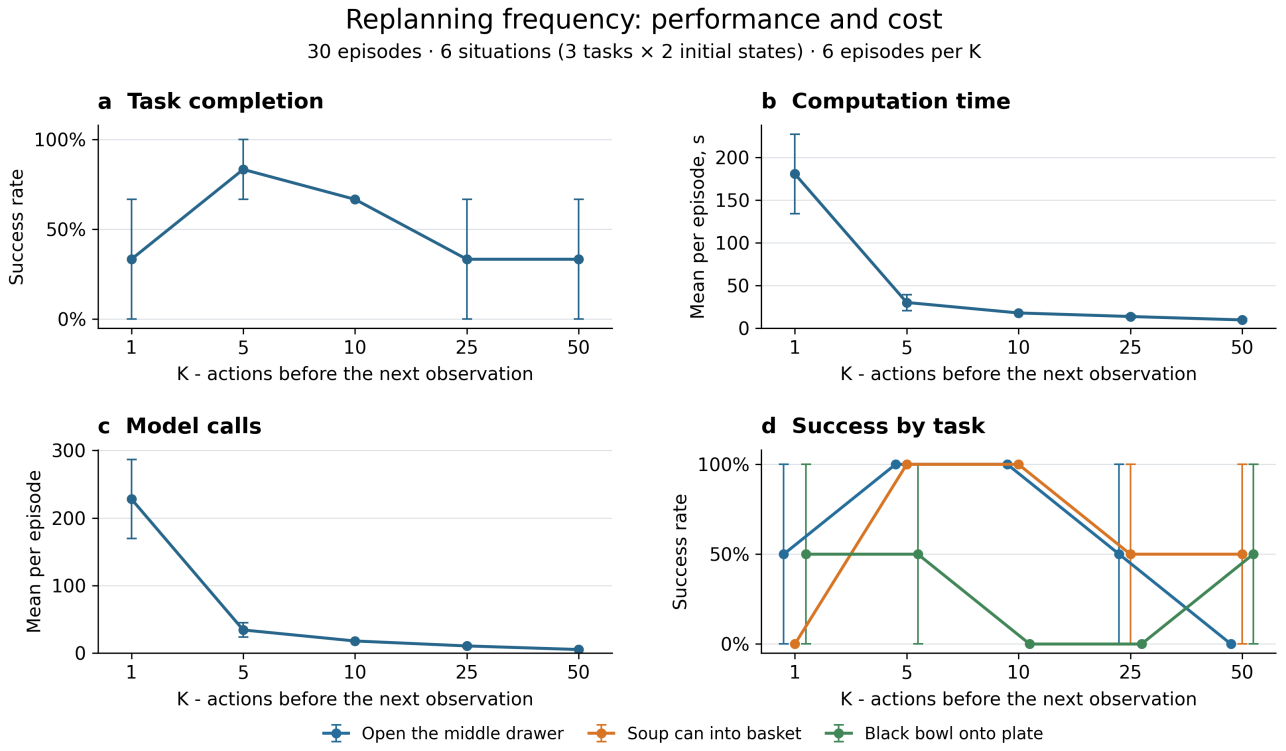


Figure 1. Success, computational cost, model calls and results by task. Lines connect the tested K values; behavior at intermediate values was not measured.

Interpreting uncertainty

Means are computed over states within each task, then over tasks with equal weights. The 95% intervals use 5,000 paired bootstrap resamples of states within each task. With only two states, these intervals are very coarse: a narrow or zero-width interval does not establish reliability or equivalence between settings. The conclusions are descriptive.

Cost, motion and errors

Values are means per episode. The number of calls depends on both K and how early the episode ends: calls = ceil(steps / K).

K	Calls	Model, s	Simulation, s	Sum, s	Episode, s
1	228.2	175.4	5.3	180.6	184.6
5	34.2	26.3	3.8	30.1	35.0
10	17.8	13.8	4.0	17.8	21.5
25	10.5	8.1	5.5	13.6	18.5
50	5.2	4.1	5.5	9.6	14.0

Entire study: 1775 model calls; 6409 steps; sum of episode durations 27.4 min. This excludes installation, weight downloads and process startup with model loading.

Model: preprocessing + inference + postprocessing, with GPU synchronization. Simulation: env.step and rendering. Sum = model + simulation. Episode time includes reset, execution, frame recording and video finalization; it excludes model loading, warmup and JSON-log writes. Simulated duration = steps / 20; this differs from wall time.

Motion proxies

K	EEF path, m	Plan revision	Replan jump	Gripper closes
1	0.89	0.382	0.406	20.7
5	1.02	0.331	0.349	5.7
10	1.19	0.462	0.488	8.2
25	1.44	0.539	0.517	5.3
50	1.32	undefined	0.437	2.8

Plan revision is the L2 difference between the new first translation vector and the next vector in the old plan; K = 50 leaves no tail. Replan jump is the L2 difference between consecutive translation commands at replanning. These are command units, not meters. A changed command alone does not demonstrate a useful correction.

Video annotation: coverage and events

K	Complete labels	Misses: sum [n]	Drops: sum [n]
1	3/6	0 [3/6]	0 [3/6]
5	5/6	0 [5/6]	0 [5/6]
10	4/6	0 [4/6]	0 [4/6]
25	3/6	0 [3/6]	1 [4/6]
50	4/6	0 [4/6]	0 [4/6]

Complete labels: 19/30 episodes. Brackets give the number of episodes with a determinate count for that event, out of six. Sums over known episodes do not imply zero events in the remaining episodes and do not estimate the event rate for the entire study. A drop can be clear even when the number of misses is not. A miss means closing the fingers without retaining the target; failures to hook a handle with an open gripper are excluded. An assistant annotated frames and logs; there was no independent human review.

Observations from paired episodes

The observations below come from discussion.json. Each comparison uses the same task, initial state and seed. Possible causes are distinguished from observed behavior.

1. Can: object loss at K=25

Soup can into basket; state=1; seed=11. K=10: success, 136 steps, 14 calls; K=25: failure, 280 steps, 12 calls.

Observation. K=10 carries the can and completes the task at 136. At K=25, the can is held at approximately 55-92, then falls outside the basket at 93-96; after the update at 100, the empty hand continues towards the basket. The gripper command remains closing during the loss.

Possible explanation. Slipping or unfavorable contact geometry are possible explanations. A longer interval leaves more motion without a new observation, but even the next plan does not recover the grasp. Contact forces were not recorded, so the exact cause is unknown.

Verifiable evidence. Two cameras: steps 55-100. steps.jsonl: grip at 85-100 ≈ +0.987…+1; fingers converge after the loss. chunks.jsonl: updates at 75 and 100. The annotation records one clear drop; the number of early missed grasps is unknown.

2. Bowl: infrequent updates can work

Black bowl onto plate; state=1; seed=11. K=5: failure, 280 steps, 56 calls; K=50: success, 191 steps, 4 calls.

Observation. K=50 carries the bowl to the plate edge at approximately 50-75; after the update at 150, it returns and adjusts the position at 160-190, succeeding at 191. K=5 interacts near the plate and tilts it, but does not reach the goal within 280 steps.

Possible explanation. Executing a long action chunk may have preserved motion consistency. One example cannot establish this: in state 0, K=5 succeeds and K=50 fails. The initial scene geometry affects the outcome.

Verifiable evidence. K=50: 4 calls, 8.24 s for model + simulation. K=5: 56 calls, 50.29 s. Video: plate contact at 107-118/199-204 for K=5; return at 160-190 for K=50. Ambiguous grasp events are not treated as zero.

3. Drawer: same success, different cost

Open the middle drawer; state=1; seed=11. K=1: success, 154 steps, 154 calls; K=10: success, 109 steps, 11 calls.

Observation. Both settings open the drawer. K=1 succeeds after 154 steps and 154 calls; K=10 after 109 steps and 11 calls. The gripper stays open: motion relies on contact with the handle rather than finger closure.

Possible explanation. Rebuilding the plan after every step may have disrupted a consistent pull. This cause is not established; the reliable measurements show only that K=1 had no outcome advantage and had a higher cost.

Verifiable evidence. summary.json and video: K=1 - 121.27 s for model + simulation; K=10 - 10.62 s. Paired initial states and observations match. Zero missed finger grasps does not imply zero failures to engage the handle.

Interpreting changes in motion

The REPLAN marker identifies when a plan is replaced. A subsequent change in direction establishes a temporal relationship, but does not by itself show that replanning caused an improvement. A causal claim would require additional controlled interventions.

correction_after_replan among episodes with complete labels: yes=4, no=2, unclear=13. A yes label requires specific steps in annotations.csv, cross-checked against chunks.jsonl.

Findings and limitations

What this study shows

1. In this series, K=5 achieved 5 successes out of 6, K=10 achieved 4/6, and K=1,25,50 achieved 2/6 each. These are results for six selected situations, not evidence of a universal optimum. For the bowl, the best interval changed: K=1/5 completed state 0, whereas only K=50 completed state 1.

2. K=1 required an average of 228.2 model calls and 180.6 s for model inference and simulation; K=10 required 17.8 calls and 17.8 s; K=50 required 5.2 calls and 9.6 s. Even with the same successful outcome for drawer state 1, K=1 made 154 calls versus 11 for K=10. More frequent feedback did not reach the goal sooner here.

3. For the can in state 1, K=10 completed the task, whereas K=25 lost the can at steps 93-96. Replanning at 100 did not lead to recovery: the empty hand continued along its path. A new observation provides an opportunity to correct an error, but the policy need not use it.

4. For the drawer in state 0 at K=25, after an unsuccessful pull at 105-135, the hand approached again at 140-200 and opened the drawer at 205-228. Replans at 125/150 coincided with the changed approach. This is an observed correction; its causal dependence on the new observation was not tested separately.

Limitations

There are only two initial situations per task and one generation seed. A difference of one episode corresponds to 50 percentage points within a task. This study illustrates behavior and cost, but does not precisely estimate the probability of success in arbitrary situations.

Three tasks from one simulator are evaluated with one checkpoint. Generalization to other models, task sets, object properties or a physical robot requires new evaluation. Contact and grasping errors may have several possible causes.

Success is LIBERO's native signal, and the episode ends immediately afterward. The can entering the target region may count as success while it is still held by the fingers. Stable placement after release and subsequent seconds of motion were not separately tested.

Calls are synchronous: physics pauses while the GPU predicts a plan. Thus, 20 Hz is the simulated control frequency. This experiment does not evaluate a physical robot under inference latency or demonstrate real-time operation.

Early termination changes path length, call count and episode duration. A shorter path does not necessarily mean better motion, and lower cost does not substitute for success. Visual annotation is subjective; unreviewed or ambiguous events must remain unknown.

Conclusion

Replanning frequency creates a trade-off between updating the observation, motion consistency, and computational cost. Across the six tested situations, K=5 produced the most successes; K=10 was close in success rate and cheaper, whereas K=50 sharply reduced the number of calls but lost some successful outcomes. K=1 did not deliver the highest success rate and was the most expensive setting. Choosing an operating K requires testing on new initial states. The current data support a non-monotonic relationship, but do not determine a universal frequency.

Possible extensions

A next step would be more independent initial states, followed by multiple seeds. Object perturbations and adaptive K require a separate, predefined protocol. These additional experiments are outside the scope of the current compact study.

Reproducibility and materials

Run created: 2026-10-17T16:57:50.172060+00:00. Device: NVIDIA GeForce RTX 2060; CUDA 12.6; Python 3.12.3; Linux. LeRobot 0.6.1; hf-libero 0.1.4; MuJoCo 3.3.2; PyTorch 2.7.1.

Model: HuggingFaceVLA/smolvla_libero
revision: 6721902bc4d61e50a3bfdb11dfb4cb626f05d102
Backbone: HuggingFaceTB/SmolVLM2-500M-Instruct
revision: 7b375e1b73b11138ff12fe22c8f2822d8fe03467

Commands from the repository root

```
bash scripts/remote_setup.sh
bash scripts/run_remote.sh configs/compact.json outputs/compact
.venv/bin/python scripts/finish_run.py outputs/compact
```

After reviewing videos and completing annotations.csv, rerun the analysis and record observations and testable hypotheses in discussion.json. PDF generation requires reportlab and pypdf.

```
.venv/bin/python -m vla_replan analyze outputs/compact --output outputs/compact/report --annotations outputs/compact/annotations.csv
.venv/bin/python scripts/build_figures.py --run outputs/compact
python scripts/build_report.py --run outputs/compact --discussion outputs/compact/discussion.json
```

Saved materials

manifest.json - conditions, versions and hashes; episodes/*/summary.json - outcomes and costs; steps.jsonl and chunks.jsonl - motion, actions and replanning steps; video.mp4 - two camera views, instruction and outcome; report/ - tables and figures; demos/ - paired K=1 and K=50 videos and motion traces; annotations.csv - visual labels; discussion.json - interpretation.

Config SHA-256: 077d7985f55beedf7f738c9dda9e9f1d226110e3d9e82f98c4c7a78970509eee
Code SHA-256: d96cfeded0dc6e1ca630d2fbdc9f23e1cdf51806bfbe14211327200da412902e
Lock SHA-256: 8574b51f0fad0b6ec40c02b3c6192da4606e30a94fd8dc1e18068b461d81212a
Discussion SHA-256: 9ddbfb61b80abe8e0e08ce46e3aae4101c892605cf73bc9f03284059ba3a788a

References

1. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics
2. SmolVLA LIBERO checkpoint, pinned revision
3. LeRobot 0.6.1: SmolVLA implementation
4. LeRobot 0.6.1: LIBERO adapter
5. LIBERO environment documentation

All numerical results in this report come from completed episodes in this study. The references above describe the model and software environment, not the outcomes measured here.